Détection de Défauts sur Cartes PCB par Deep Learning

Architecture YOLOv8 — Transfer Learning
Tâche de détection d'objets

Résumé exécutif

Ce rapport présente l'étude d'un modèle YOLOv8n entraîné par transfer learning pour la détection automatique de 6 types de défauts sur cartes de circuits imprimés (PCB). Le modèle atteint une **mAP@0.5 de 0.985** et une **Precision de 0.971** sur le jeu de test, démontrant d'excellentes capacités de généralisation.

Cours	Deep Learning for Computer Vision
Année	2025 / 2026
Auteurs	Riad SAHRANE Abdourahmane TIMERA
Modèle	YOLOv8n (Ultralytics)
Dataset	PCB Defect Dataset — 1 500 images, 6 classes

Table des matières

1 Introduction **2**
1.1 Problématique . . . 2

2 Jeu de données **2**
2.1 Description générale . . . 2
2.2 Classes de défauts . . . 3
2.3 Distribution des classes . . . 3
2.4 Stratégie de split train / validation / test . . . 3

3 Architecture choisie : YOLOv8 **4**
3.1 Famille YOLO et choix de YOLOv8 . . . 4
3.2 Variante YOLOv8n (nano) . . . 4
3.3 Architecture du réseau . . . 5
3.4 Principe de la détection . . . 5

4 Méthodologie d'entraînement **5**
4.1 Transfer Learning (Fine-tuning) . . . 5
4.2 Hyperparamètres et justifications . . . 6
4.3 Préparation des données . . . 6
4.4 Résultats d'entraînement . . . 6

5 Résultats et évaluation **7**
5.1 Métriques d'évaluation . . . 7
5.2 Résultats quantitatifs sur le jeu de test . . . 7
5.3 Résultats par classe sur le jeu de test . . . 8
5.4 Matrice de confusion . . . 9
5.5 Courbes Precision-Recall et F1 . . . 9
5.6 Résultats qualitatifs . . . 10
5.7 Analyse des erreurs . . . 10
5.7.1 Cas difficile : image `12000593.jpg` . . . 11
5.7.2 Interprétation . . . 11
5.8 Impact du seuil de confiance . . . 12
5.9 Temps d'inférence . . . 12

6 Discussion **12**
6.1 Points forts du modèle . . . 12
6.2 Limitations . . . 13
6.3 Pistes d'amélioration . . . 13

7 Conclusion **13**

1. Introduction

La détection automatique de défauts sur cartes de circuits imprimés (PCB) est un enjeu industriel majeur dans le secteur de l'électronique. Les approches traditionnelles de contrôle qualité reposent sur l'inspection visuelle manuelle, coûteuse et sujette à des erreurs humaines. L'essor du deep learning, et plus particulièrement des réseaux de neurones convolutifs (CNN), offre aujourd'hui des solutions robustes et rapides.

Objectif du projet : concevoir et évaluer un pipeline complet de détection de défauts PCB basé sur l'architecture YOLOv8, depuis la préparation des données jusqu'à l'évaluation quantitative et qualitative sur un jeu de test indépendant.

1.1. Problématique

Les défauts PCB se manifestent sous des formes visuellement diverses : courts-circuits, connexions ouvertes, perforations, bavures de cuivre. . .Un détecteur efficace doit être capable de :

— Localiser précisément chaque défaut (bounding box)

— Classifier correctement le type de défaut parmi 6 catégories

— Fonctionner en temps réel pour s'intégrer dans une chaîne industrielle

2. Jeu de données

2.1. Description générale

Le dataset utilisé est le **PCB Defect Dataset**, un jeu de données d'images de cartes de circuits imprimés réelles annotées avec des bounding boxes au format XML converti en format `xyxycls` (coordonnées pixels).

Table 1 – Caractéristiques générales du dataset

Propriété	**Valeur**
Nombre total d'images	1 500
Résolution des images	640 × 640 pixels
Nombre de classes	6
Nombre total de bounding boxes	10 013
Moyenne bbox / image	6.68
Médiane bbox / image	7
Maximum bbox / image	15
Format des annotations	xyxycls (pixels) → converti en YOLO normalisé

2.2. Classes de défauts

Les 6 classes représentent des défauts électroniques distincts, décrits dans le Tableau 2.

Table 2 – Description des 6 classes de défauts PCB

ID	Classe	Description
0	`open`	Connexion interrompue — trace conductrice coupée
1	`short`	Court-circuit — deux pistes conductrices reliées involontairement
2	`mousebite`	Entaille en demi-lune sur le bord d'une piste
3	`spur`	Excroissance de cuivre parasite sur une piste
4	`copper`	Résidu de cuivre non gravé hors des pistes
5	`pine_hole`	Trou de perçage anormal (diamètre ou position incorrects)

2.3. Distribution des classes

Table 3 – Répartition des bounding boxes par classe

Classe	Nb bbox	Part (%)
open	1 942	19.4%
short	1 506	15.0%
mousebite	1 622	16.2%
spur	1 655	16.5%
copper	1 644	16.4%
pine_hole	1 644	16.4%
Total	**10 013**	**100%**

Observation : Les classes sont relativement équilibrées (entre 15% et 19.4% chacune). Cet équilibre est favorable à l'entraînement car il réduit le risque de biais du modèle vers une classe dominante.

2.4. Stratégie de split train / validation / test

Le dataset a été divisé aléatoirement (graine fixe `seed=42`) selon le ratio **70** / **15** / **15** :

Table 4 – Répartition du dataset

Split	Images	Proportion	Rôle
Train	1 050	70%	Apprentissage des paramètres
Validation	225	15%	Sélection du meilleur modèle (early stopping)
Test	225	15%	Évaluation finale honnête

Ce ratio est standard pour des datasets de taille intermédiaire (1 000–5 000 images). La taille du jeu de validation (225 images) est suffisante pour obtenir des estimations stables des métriques epoch par epoch, et le jeu de test n'a jamais été utilisé pendant la phase d'entraînement ou de sélection du modèle.

3. Architecture choisie : YOLOv8

3.1. Famille YOLO et choix de YOLOv8

YOLO (*You Only Look Once*) est une famille de réseaux de neurones convolutifs dédiée à la détection d'objets en temps réel. Contrairement aux approches à deux étapes (Faster R-CNN), YOLO résout la détection en un seul passage (*single-shot*) du réseau, ce qui lui confère une vitesse d'inférence très élevée.

YOLOv8 [1], publié par Ultralytics en 2023, représente l'état de l'art de cette famille. Par rapport aux versions précédentes, il introduit :

— Une tête de détection *anchor-free* (plus flexible)

— Un backbone CSPDarknet amélioré

— Un mécanisme d'attention C2f (Cross Stage Partial with 2 convolutions)

— De meilleures performances sur les petits objets

3.2. Variante YOLOv8n (nano)

Parmi les variantes disponibles (n, s, m, l, x), nous avons sélectionné **YOLOv8n** (*nano*) pour les raisons suivantes :

Table 5 – Comparaison des variantes YOLOv8 sur COCO val2017

Modèle	Paramètres	GFLOPs	mAP@0.5 :0.95	Vitesse GPU
YOLOv8n	3.2M	8.7	37.3	**0.99 ms**
YOLOv8s	11.2M	28.6	44.9	1.20 ms
YOLOv8m	25.9M	78.9	50.2	1.83 ms
YOLOv8l	43.7M	165.2	52.9	2.39 ms

Dans notre contexte (dataset spécialisé PCB, 6 classes, transfer learning depuis COCO), YOLOv8n est **suffisant** car la tâche est plus simple que la détection généraliste sur 80 classes de COCO. De plus, la contrainte de ressources GPU (entraînement sur Google Colab T4) favorise l'utilisation d'un modèle léger.

3.3. Architecture du réseau

YOLOv8n est composé de **130 couches**, **3 012 018 paramètres** et requiert **8.2 GFLOPs** par inférence. Il s'organise en trois blocs :

TABLE 6 – Structure de l'architecture YOLOv8n

Bloc	Composant	Rôle
Backbone	CSPDarknet + C2f	Extraction de features à différentes résolutions
Neck	FPN + PAN	Fusion multi-échelle (petits et grands objets)
Head	Anchor-free	Prédiction bbox, confiance, classification

3.4. Principe de la détection

À l'inférence, YOLOv8 divise l'image d'entrée (640×640) en une grille et prédit, pour chaque cellule, un ensemble de bounding boxes avec :

— Les coordonnées (x_c, y_c, w, h) normalisées de la boîte

— Un score de confiance p_{obj}

— Un vecteur de probabilités de classe $p(c_i)$

La détection finale est obtenue après un filtrage par seuil de confiance et *Non-Maximum Suppression* (NMS) pour éliminer les détections redondantes.

4. Méthodologie d'entraînement

4.1. Transfer Learning (Fine-tuning)

Plutôt qu'un entraînement depuis zéro (*from scratch*), nous avons appliqué une stratégie de **transfer learning** : le modèle est initialisé avec les poids pré-entraînés sur **COCO** (80 classes, 118 000 images) puis fine-tuné sur notre dataset PCB.

Avantages du transfer learning :

— Les features bas-niveau (contours, textures) déjà apprises sur COCO sont directement réutilisables pour les défauts PCB

— Convergence plus rapide (moins d'epochs nécessaires)

— Meilleure généralisation avec peu de données

4.2. Hyperparamètres et justifications

Table 7 – Hyperparamètres d'entraînement

Hyperparamètre	Valeur	Justification
Modèle de base	yolov8n.pt	Léger, adapté à un GPU T4 et un dataset de taille modérée
Taille d'image	640	Standard YOLO — bon compromis précision / vitesse
Batch size	16	Équilibre entre stabilité du gradient et mémoire GPU
Epochs max	40	Suffisant pour converger sur notre dataset
Patience (early stopping)	10	Arrêt automatique si pas d'amélioration sur 10 epochs
Optimiseur	SGD (auto)	Défaut Ultralytics, robuste pour la détection
Learning rate	0.01 (auto)	Valeur par défaut avec scheduler cosine
Augmentations	Mosaic, flip, HSV	Augmentations intégrées YOLOv8

4.3. Préparation des données

Les annotations originales étaient au format `xyxycls` (coordonnées pixels absolues). Elles ont été converties au format YOLO normalisé selon :

$$x_c = \frac{x_1 + x_2}{2W}, \quad y_c = \frac{y_1 + y_2}{2H}, \quad w = \frac{x_2 - x_1}{W}, \quad h = \frac{y_2 - y_1}{H} \tag{1}$$

où $W = H = 640$ pixels. Les identifiants de classes ont été remappés de [1..6] vers [0..5] (convention YOLO).

4.4. Résultats d'entraînement

L'early stopping a déclenché l'arrêt à l'**epoch 29** (patience = 10, meilleur modèle à l'epoch 19).

Table 8 – Métriques sur le split **validation** — meilleure epoch

Split	Precision	Recall	mAP@0.5	mAP@0.5 :0.95
Validation	0.973	0.955	0.982	0.765

TABLE 9 – Résultats par classe sur le split validation

Classe	mAP@0.5	mAP@0.5 :0.95
open	0.977	0.691
short	0.971	0.668
mousebite	0.980	0.740
spur	0.982	0.753
copper	0.990	0.880
pine_hole	0.993	0.856

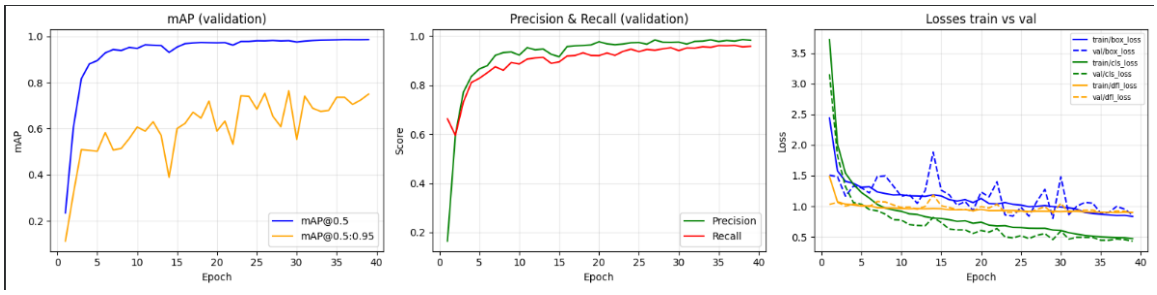

FIGURE 1 – Courbes d'entraînement — mAP@0.5, Precision, Recall, Losses train vs validation

5. Résultats et évaluation

5.1. Métriques d'évaluation

Les métriques utilisées sont les métriques standard de la détection d'objets :

— **Precision** : parmi toutes les détections, quelle fraction est correcte $P = \frac{TP}{TP+FP}$
— **Recall** : parmi tous les objets réels, quelle fraction est détectée $R = \frac{TP}{TP+FN}$
— **mAP@0.5** : mean Average Precision avec seuil IoU = 0.5
— **mAP@0.5 :0.95** : mAP moyen sur seuils IoU de 0.5 à 0.95 (métrique COCO)

Une détection est considérée vraie positive si son IoU avec la boîte GT dépasse le seuil et si la classe est correcte.

5.2. Résultats quantitatifs sur le jeu de test

Split	Precision	Recall	mAP@0.5	mAP@0.5 :0.95
Validation	0.973	0.955	0.982	0.765
Test	**0.971**	**0.954**	**0.985**	**0.777**

Les métriques sur le jeu de test sont quasi identiques à celles de la validation, ce qui confirme l'**absence d'overfitting** et une bonne capacité de généralisation du modèle.

5.3. Résultats par classe sur le jeu de test

TABLE 10 – Performances par classe — jeu de test (225 images, ≈1 504 instances)

Classe	Instances	mAP@0.5 :0.95	Difficulté
copper	–	**0.880**	Faible
pine_hole	–	**0.856**	Faible
spur	–	0.753	Modérée
mousebite	–	0.740	Modérée
open	–	0.691	Élevée
short	–	0.668	Élevée
Moyenne		**0.777**	

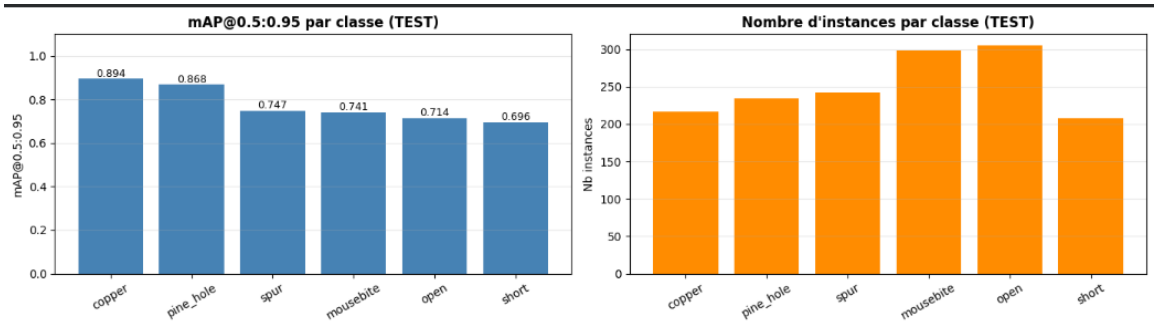

FIGURE 2 – Performances par classe — jeu de test

5.4. Matrice de confusion

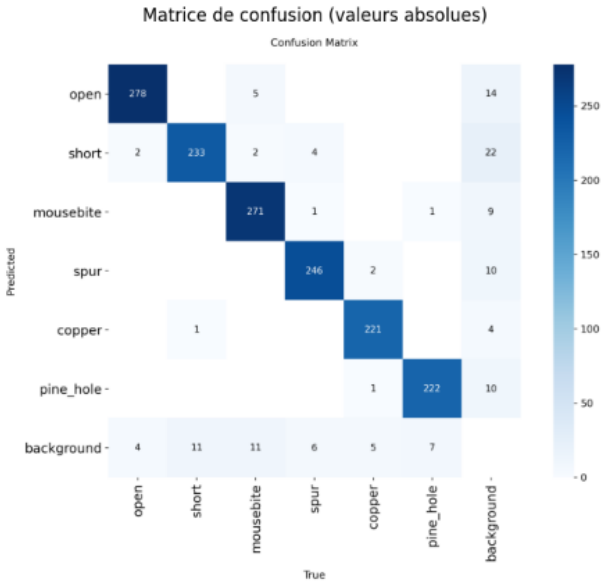

FIGURE 3 – Matrice de confusion sur le jeu de test (normalisée et valeurs absolues)

La diagonale quasi parfaite de la matrice normalisée confirme que le modèle classe correctement la grande majorité des défauts. Les légères confusions observées se concentrent entre les classes `open` et `short`, dont les motifs visuels sont proches.

5.5. Courbes Precision-Recall et F1

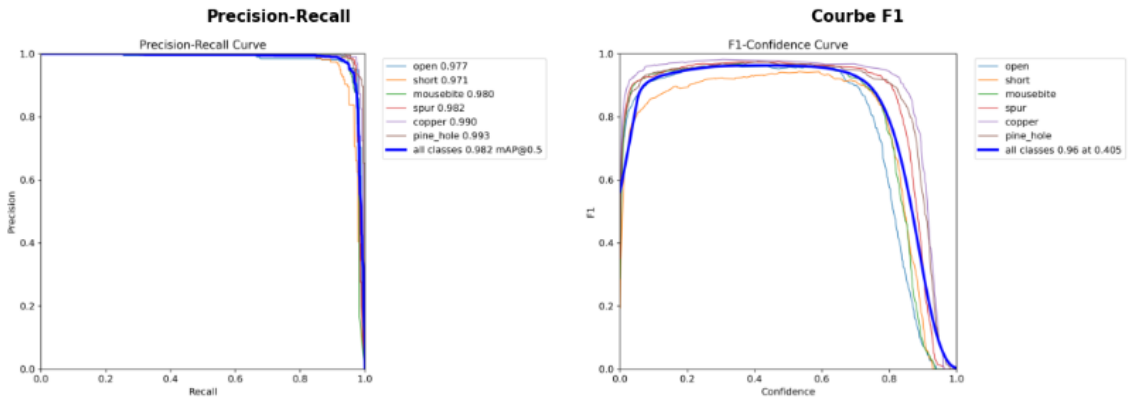

FIGURE 4 – Courbes de performance sur le jeu de test

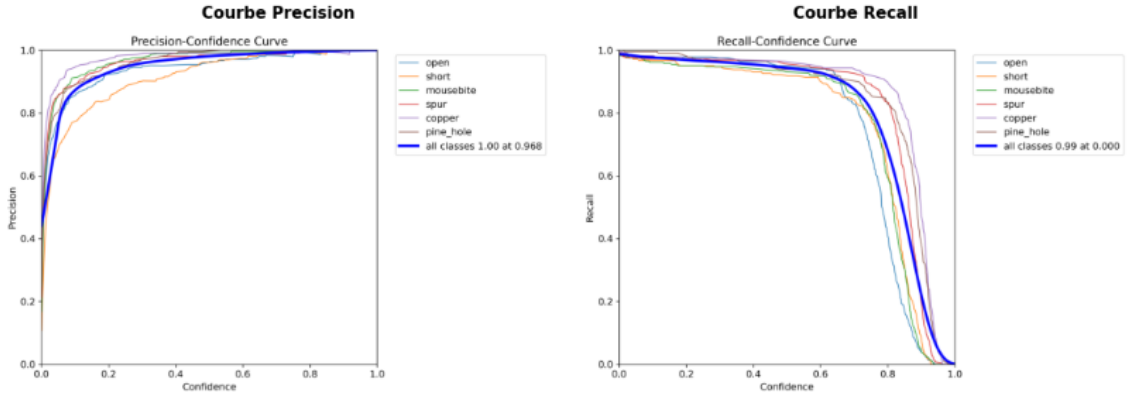

FIGURE 5 – Courbes de performance sur le jeu de test

La courbe PR présente une aire très élevée ($mAP@0.5 = 0.985$), ce qui indique que le modèle maintient à la fois une précision et un rappel élevés sur l'ensemble du spectre des seuils de confiance.

5.6. Résultats qualitatifs

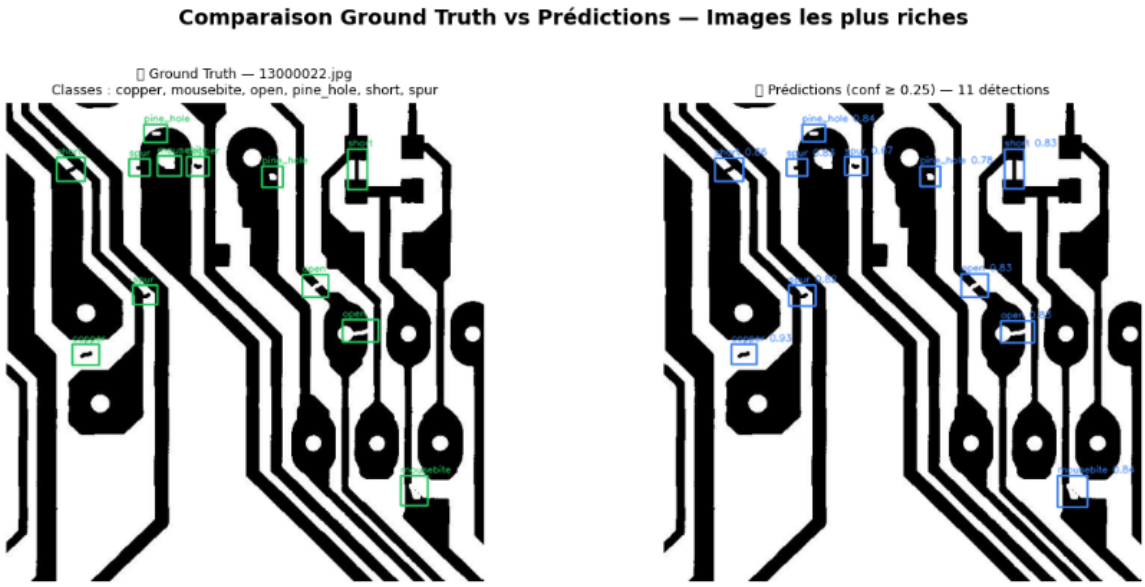

FIGURE 6 – Comparaison Ground Truth vs Prédictions sur des images à 6 classes (jeu de test)

Les visualisations montrent une excellente correspondance entre les boîtes prédites et les annotations réelles, même sur des images denses contenant 10 à 12 objets.

5.7. Analyse des erreurs

5.7.1. Cas difficile : image `12000593.jpg`

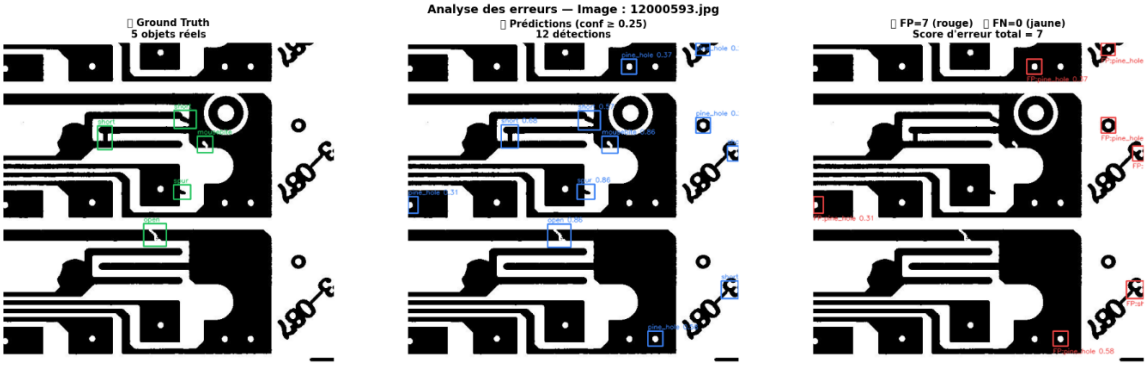

FIGURE 7 – Analyse des erreurs — image `12000593.jpg` (FP=7, FN=0)

5.7.2. Interprétation

L'analyse des erreurs révèle que les faux positifs ne sont **pas des erreurs aléatoires**. Le modèle détecte les **numéros de série et marquages sérigraphiques** (ex : `12000593`) comme des défauts `pine_hole` car ces petits caractères imprimés ressemblent visuellement à des perforations.

De même, sur l'image `13000026.jpg`, le modèle génère des faux positifs sur les **connecteurs en peigne** : les motifs répétitifs créent une ambiguïté visuelle forte avec des défauts `open` ou `short`.

Limite identifiée : le modèle travaille sur l'image PCB brute sans distinguer les zones fonctionnelles des zones de marquage. Il s'agit d'une limitation structurelle du pipeline actuel.

Solutions envisageables :

- **Pré-segmentation** : détecter et masquer les zones de texte/sérigraphie avant la détection de défauts [3]
- **Augmentation ciblée** : inclure des exemples négatifs de marquages sérigraphiques et de connecteurs sains dans l'entraînement
- **Post-traitement géométrique** : filtrer les détections dans les zones de motifs répétitifs (analyse de texture, FFT)
- **Affinement du dataset** : annoter explicitement les marquages comme classe *background*

5.8. Impact du seuil de confiance

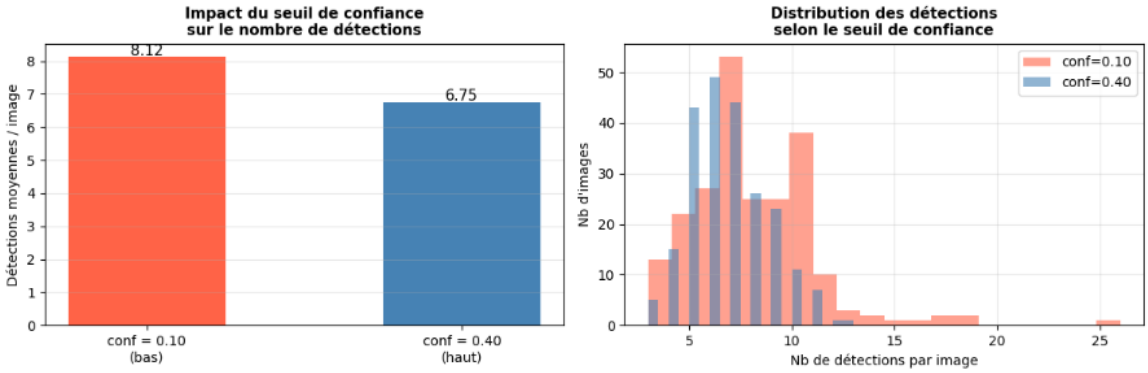

FIGURE 8 – Impact du seuil de confiance sur le nombre de détections

TABLE 11 – Comparaison selon le seuil de confiance

Seuil conf	Total détections	Moy./image	Min	Max
0.10	1 828	8.12	3	26
0.40	1 519	6.75	3	13

Pour une application industrielle de contrôle qualité PCB, on préférera un **seuil bas (≈0.25)** afin de ne manquer aucun défaut, quitte à avoir quelques faux positifs vérifiés manuellement [3, 2].

5.9. Temps d'inférence

TABLE 12 – Temps d'inférence moyen (50 mesures)

Matériel	Temps moyen	Images/seconde
CPU (Intel Xeon, Colab)	287 ms	≈ 3
GPU T4 (Colab, entraînement)	≈ 7 ms	≈ 130

Avec un GPU dédié (NVIDIA T4 ou Jetson), YOLOv8n peut traiter plus de **130 images/seconde**, ce qui est largement suffisant pour une intégration dans une chaîne de production industrielle [1].

6. Discussion

6.1. Points forts du modèle

— **Performances élevées :** mAP@0.5 = 0.985 sur le jeu de test, supérieur aux approches classiques de traitement d'images

— **Généralisation :** écart quasi nul entre validation et test (pas d'overfitting)
— **Vitesse :** compatible avec un déploiement industriel en temps réel (GPU)
— **Robustesse :** excellent rappel (0.954) sur des images denses à 6+ objets

6.2. Limitations

— **Faux positifs sur marquages sérigraphiques :** le modèle ne distingue pas les zones de texte des zones de défauts
— **Classes difficiles :** `open` et `short` ont les scores les plus faibles (mAP@0.5 :0.95 $<$ 0.70) en raison de leur similarité visuelle
— **Expérience unique :** faute de GPU disponible, une seule architecture (YOLOv8n) a pu être entraînée

6.3. Pistes d'amélioration

1. Entraîner **YOLOv8s** ou YOLOv8m avec plus d'epochs pour un gain de précision
2. Ajouter une étape de **pré-segmentation** des zones non-défectueuses
3. Augmenter le dataset sur les classes difficiles (`open`, `short`)
4. Appliquer de l'augmentation de données plus agressive (rotation, bruit, flou)
5. Explorer des architectures alternatives : RT-DETR, YOLOv9

7. Conclusion

Ce projet a permis de concevoir et d'évaluer un pipeline complet de détection automatique de défauts sur cartes PCB, basé sur l'architecture **YOLOv8n** entraînée par **transfer learning** depuis les poids COCO.

Le modèle final atteint des performances très élevées sur le jeu de test (**mAP@0.5 = 0.985**, Precision = 0.971, Recall = 0.954) et démontre une excellente capacité de généralisation, avec des métriques quasi identiques entre validation et test.

L'analyse des erreurs a mis en évidence une **limite structurelle importante** : le modèle génère des faux positifs sur les marquages sérigraphiques et les connecteurs répétitifs. Cette observation ouvre la voie à des améliorations ciblées, notamment par pré-segmentation des zones fonctionnelles.

En termes de déploiement, YOLOv8n s'avère être un choix pertinent pour une intégration industrielle grâce à sa légèreté (3M paramètres, 8.2 GFLOPs) et sa vitesse ($\approx$130 images/seconde sur GPU T4).

Références

[1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics

[2] Redmon, J., & Farhadi, A. (2018). *YOLOv3 : An Incremental Improvement*. arXiv preprint arXiv :1804.02767.

[3] Ding, R., Dai, L., Li, G., & Liu, H. (2019). *TDD-Net : A Tiny Defect Detection Network for Printed Circuit Boards*. IEEE Access, 7, 96, 139–96, 148.

[4] Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). *Feature Pyramid Networks for Object Detection*. In Proceedings of CVPR.

[5] Wang, S., Wu, X., He, Y., & Tang, H. (2020). *An Improved Faster RCNN Model for Defect Recognition of Printed Circuit Board*. Journal of Physics : Conference Series.